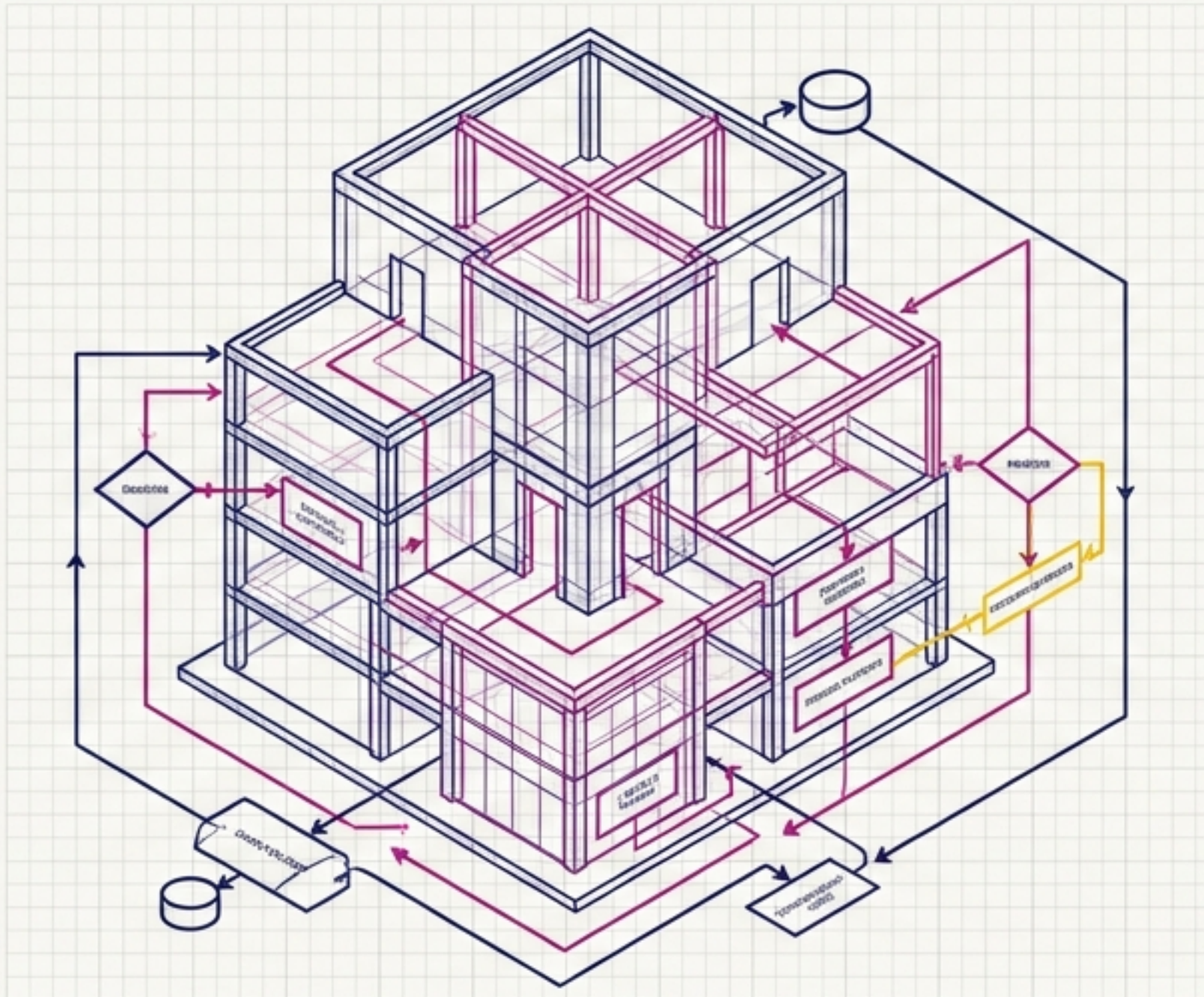


A Arquitetura dos Laços

Do Conceito Teórico ao Código: Construindo Sistemas com Estruturas de Repetição

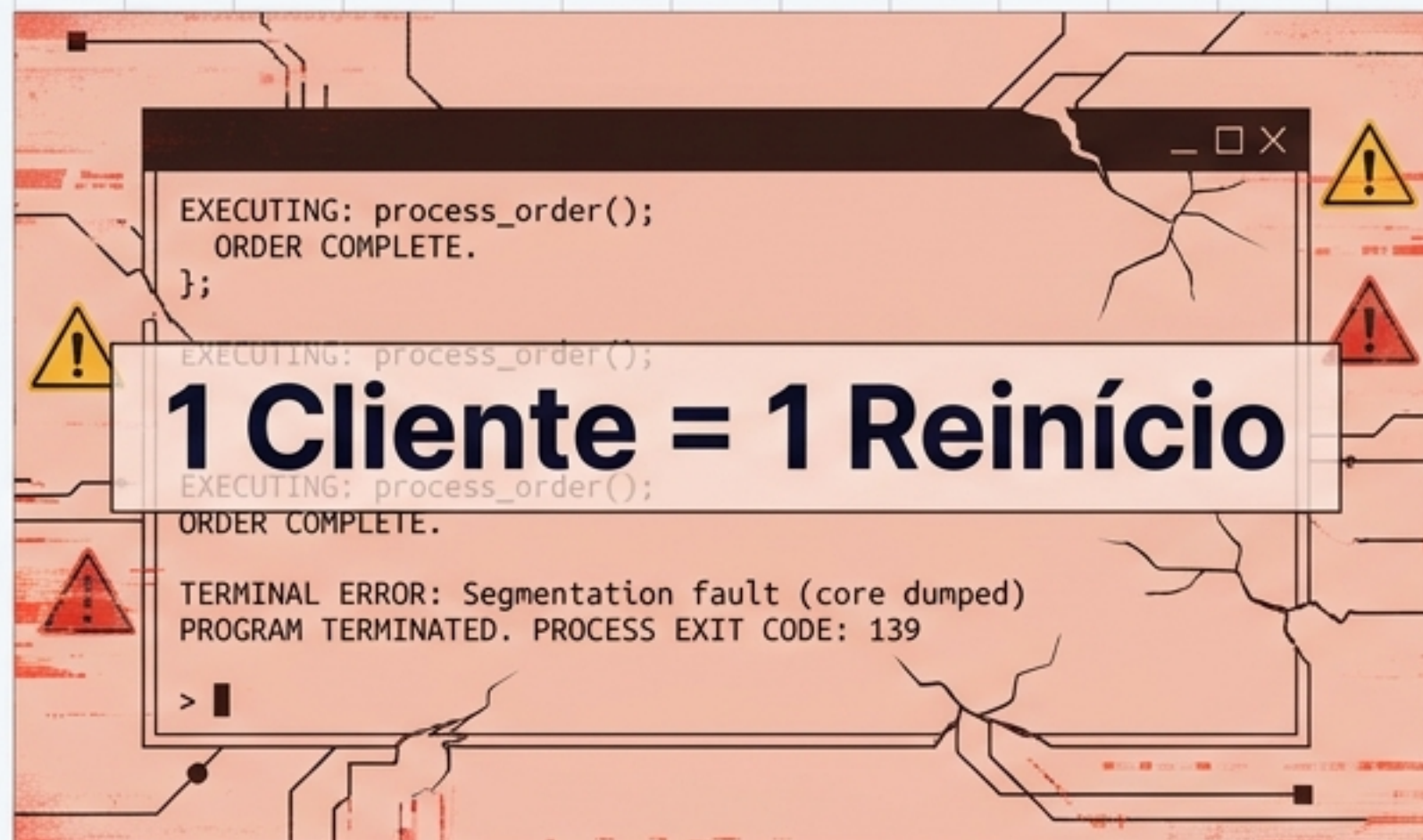
Uma análise profunda de enquanto, repita...ate e para no VisuAlg.



```
// Loop: logic nom enquanto
enquanto i = limite;
    limite <- i
    i++a = i;
enquanto

// Loop: logic e no errore:
repita
    lomio <- enquanto;
    limite <= i+0;
ate limite;

// Comonem primiarisados:
para (ia i = 0; < limite; i++)
    limite = 2;
    i += i;
end
```



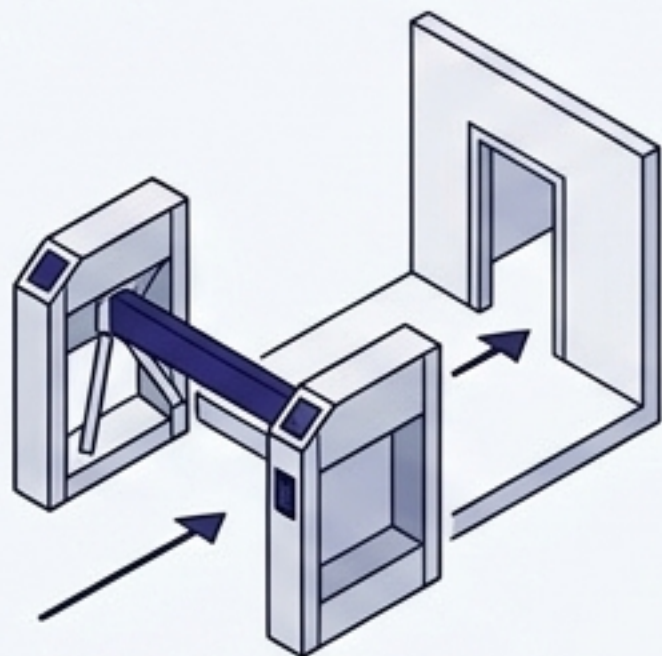
O Desafio do Atendimento Contínuo

Como manter um sistema digital (como o menu de uma lanchonete) rodando ininterruptamente para **100 clientes** sem precisar reiniciar o programa a cada pedido?

A resposta está na **Lógica de Repetição**. **Laços** (**loops**) são a base arquitetônica que impede um programa de morrer após a primeira instrução, criando ambientes interativos e persistentes.

O Nosso Kit de Ferramentas

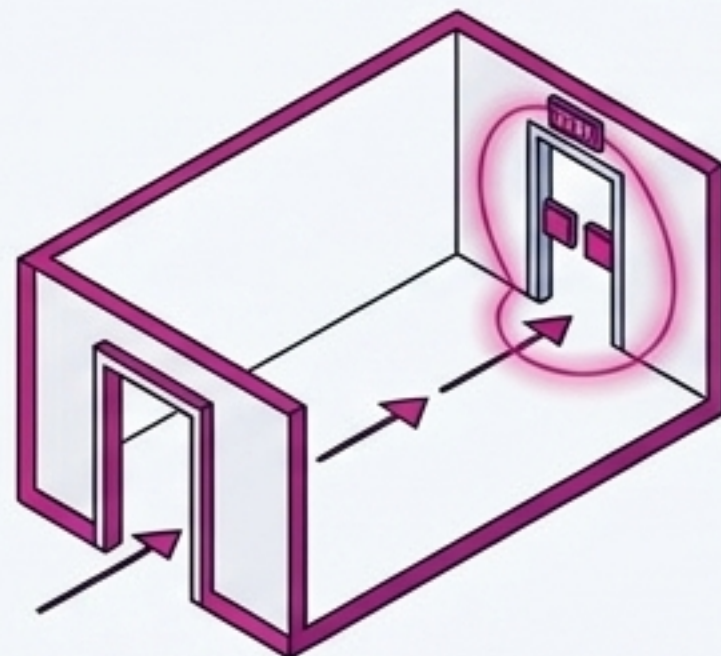
A Verificação Prévia



Executa o bloco apenas se a condição inicial for verdadeira. Ideal para incertezas.

```
enquanto ... faça
```

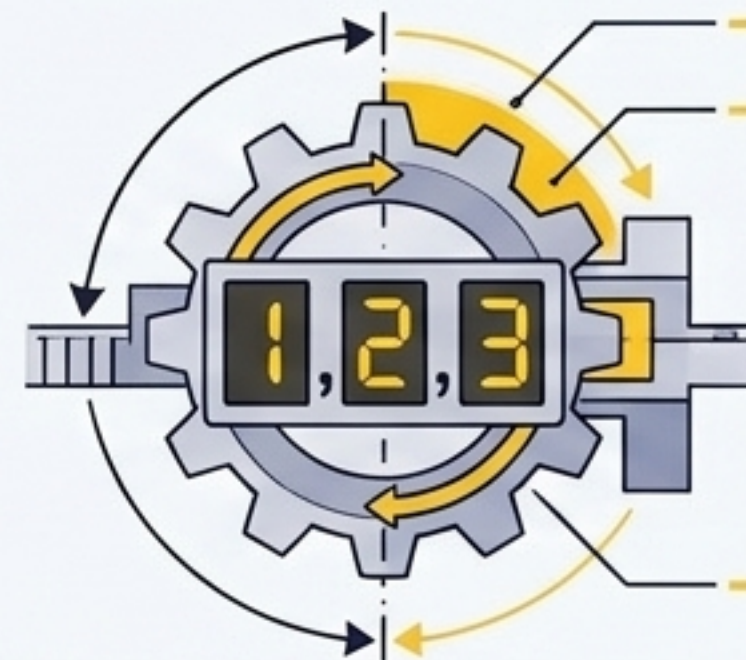
A Execução Garantida



Executa o bloco pelo menos uma vez e testa no final. A ferramenta ideal para menus.

```
repita ... ate
```

A Contagem Exata



Possui um contador integrado. Perfeito quando sabemos exatamente o número de repetições.

```
para ... de ... ate
```

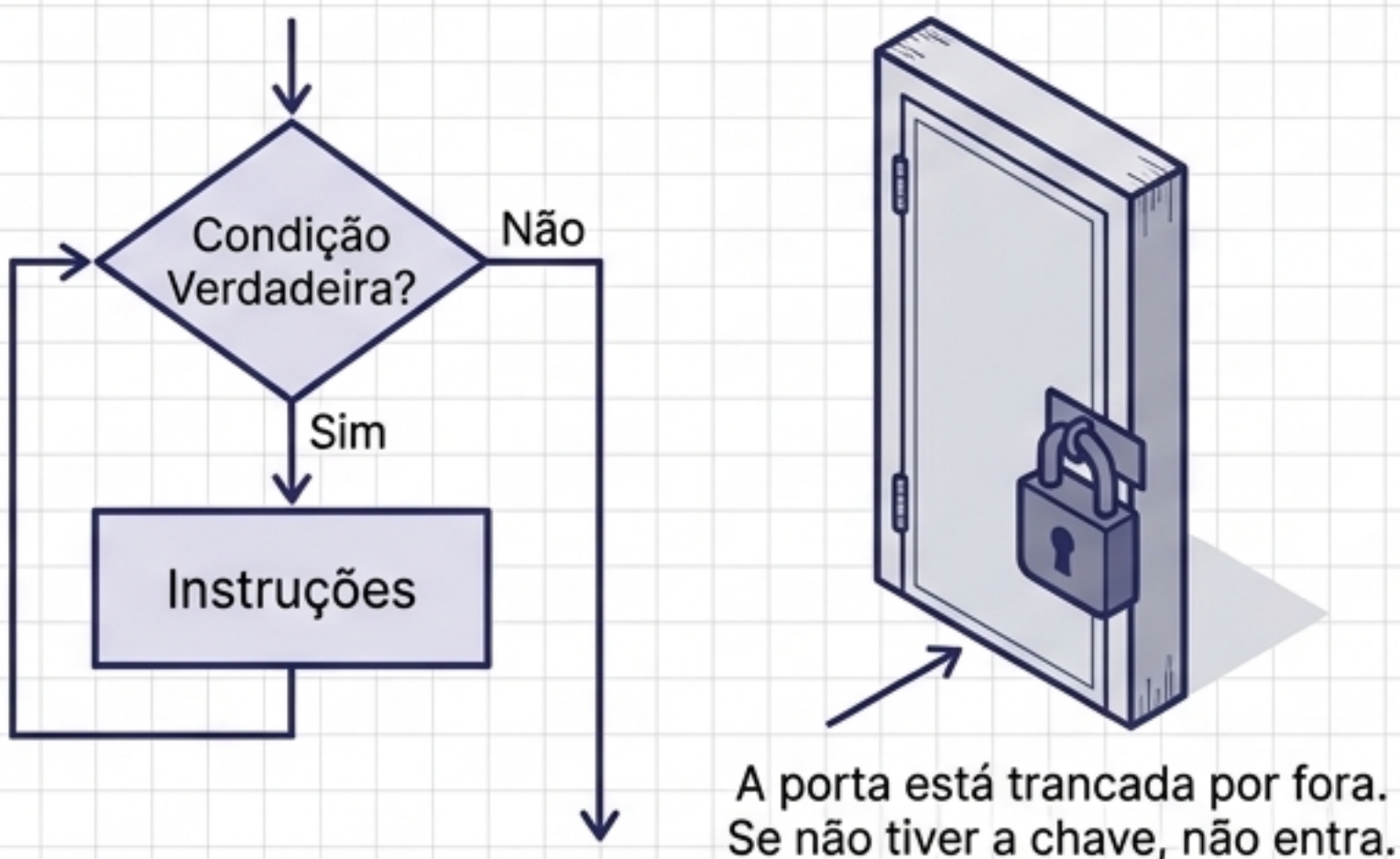
Matriz de Decisão: Qual Laço Escolher?

Dimensão	para	enquanto	repita...ate
Número de repetições conhecido?	✓ Sim	✗ Não	✗ Não
Onde ocorre o teste lógico?	Início	Início (Pré-testado)	Fim (Pós-testado)
Risco de executar ZERO vezes?	Sim	Sim	Não (Sempre executa ao menos 1 vez)
Melhor Caso de Uso	Fatoriais, varrer vetores	Somar valores até digitar 0	Menus interativos e validação de entrada

A Anatomia do Fluxo: Pré-testado vs. Pós-testado

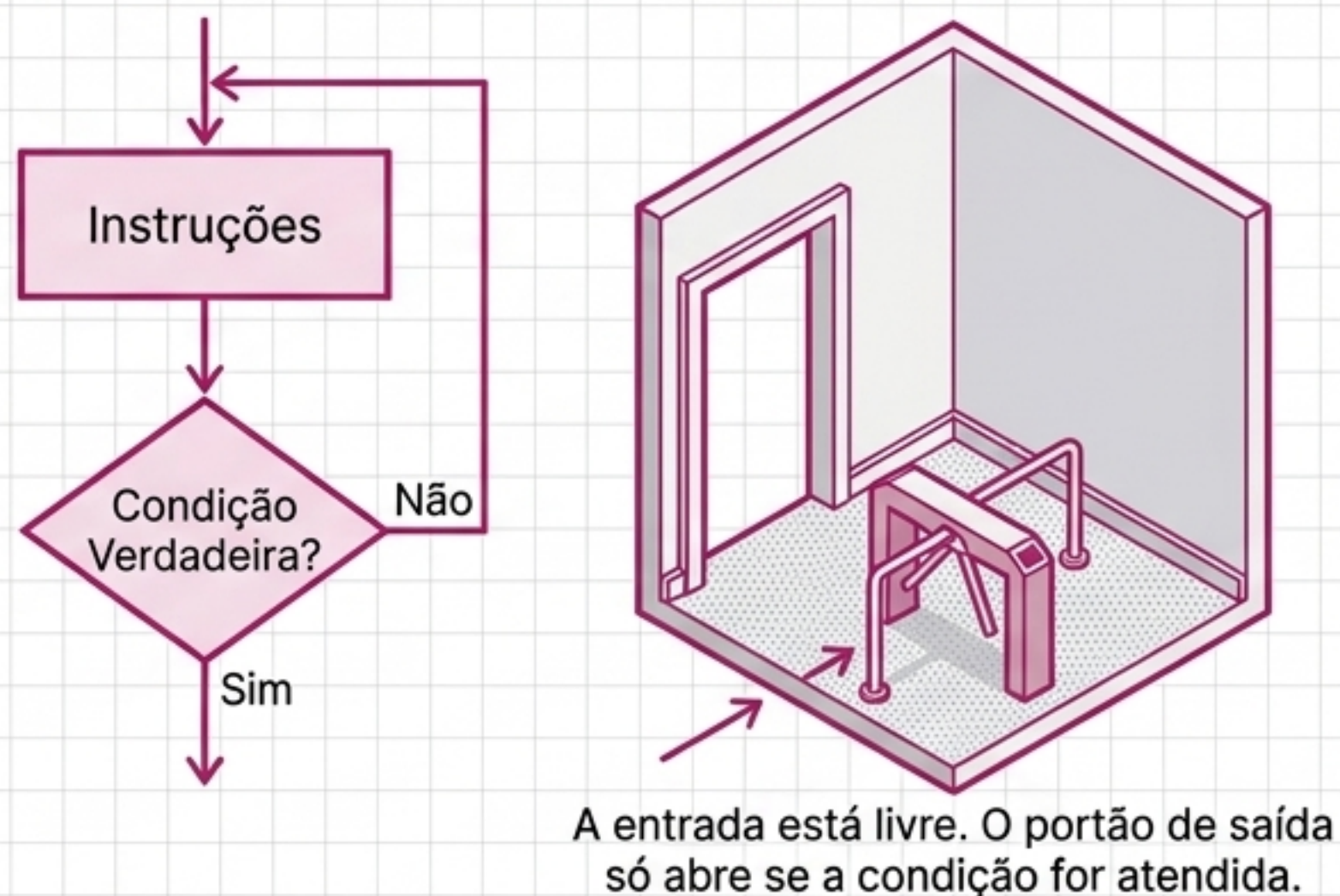
enquanto (Pré-testado)

- A condição é a chave da porta.
- Se for falsa logo de cara, as instruções internas nunca são executadas.



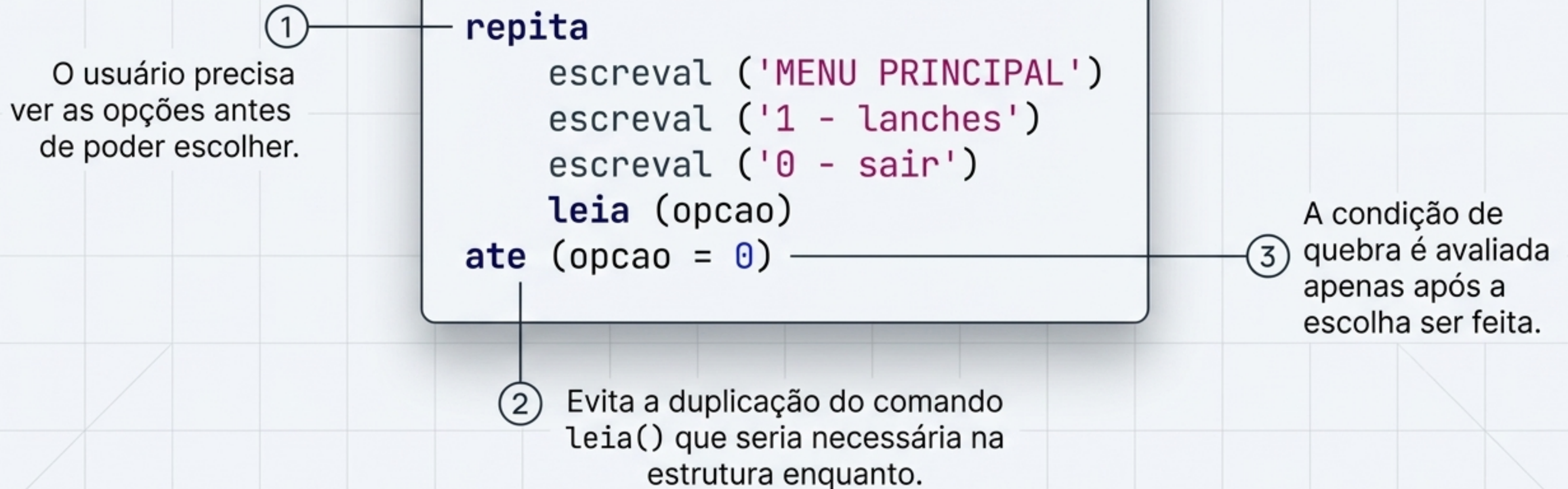
repita...ate (Pós-testado)

- A instrução ocorre primeiro.
- Garante que o usuário interaja com o código ao menos uma vez antes da avaliação. O teste decide se ele volta ou sai.



A Escolha Certa para Menus Interativos

Por que a estrutura repita...ate é a campeã indiscutível para o sistema da nossa Lanchonete?



A Planta Baixa do Algoritmo da Lanchonete

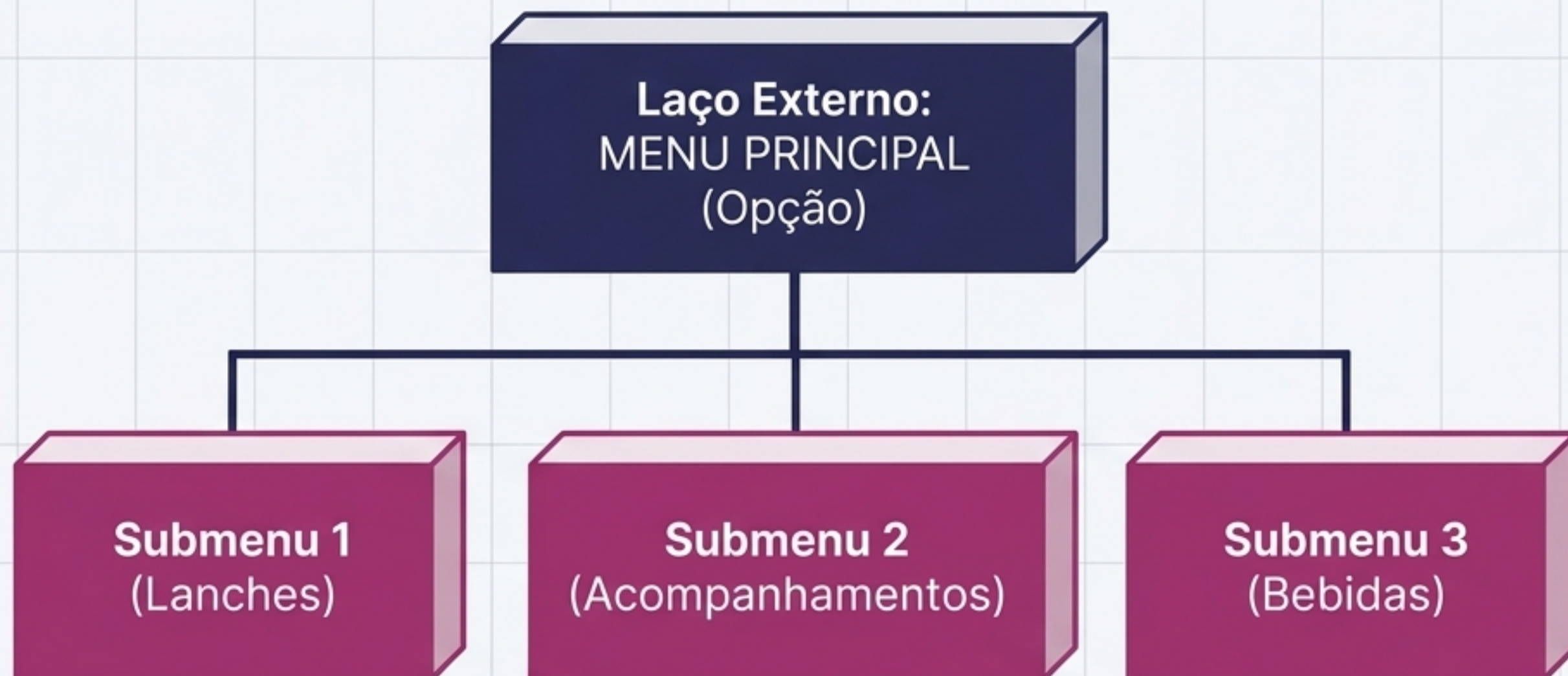
Sistemas complexos são construídos colocando laços dentro de laços (Laços Aninhados).

Nível 1 (O saguão):

O Menu Principal controla para qual corredor o cliente vai.

Nível 2 (Os balcões):

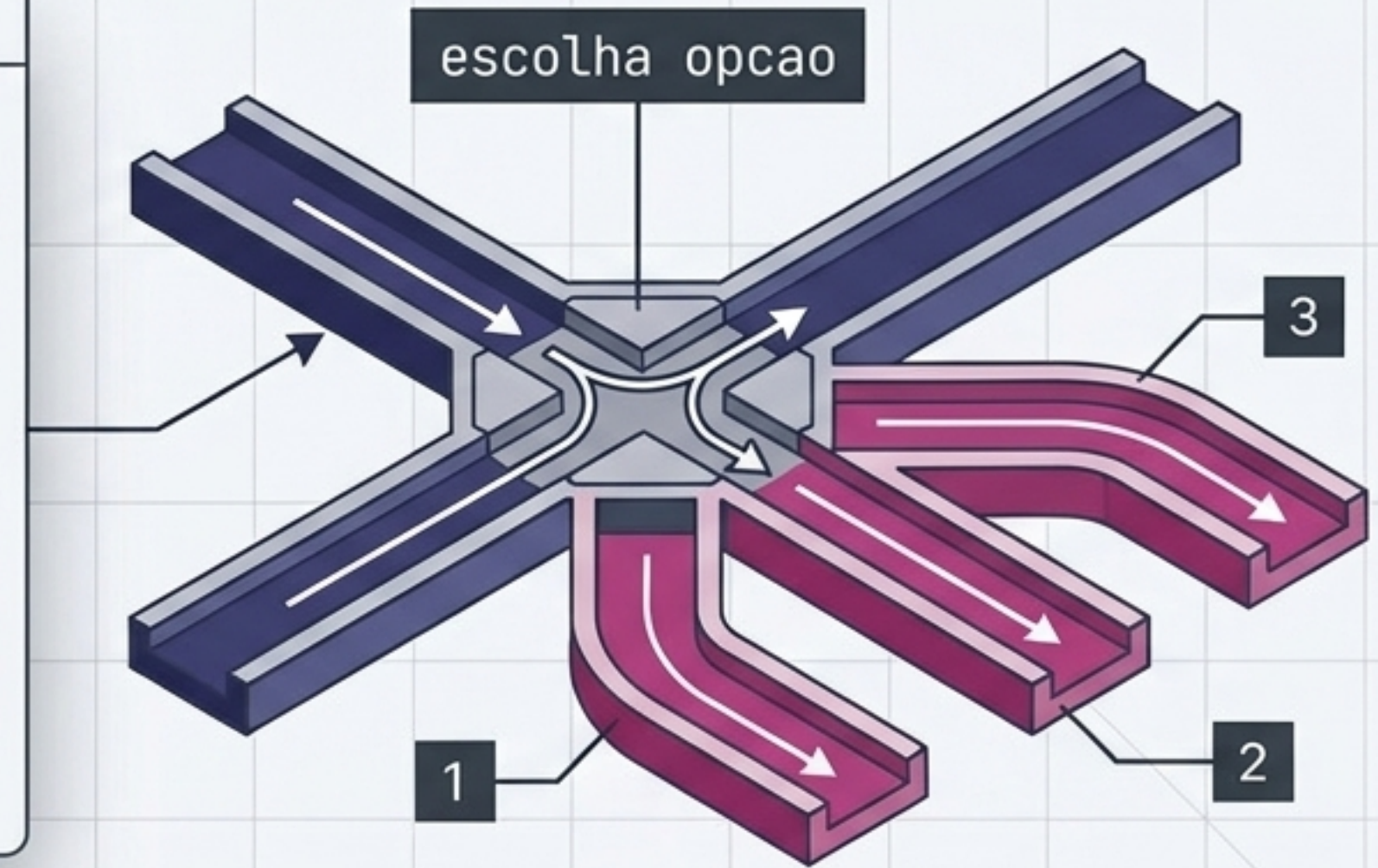
Os Submenus controlam os produtos específicos. O cliente fica no balcão até terminar de escolher, depois volta ao saguão.



O Laço Externo: A Porta de Entrada

A variável **opcao** é o **volante** do sistema. O laço mantém a lanchonete aberta, e a estrutura **escolha direciona** o fluxo do cliente.

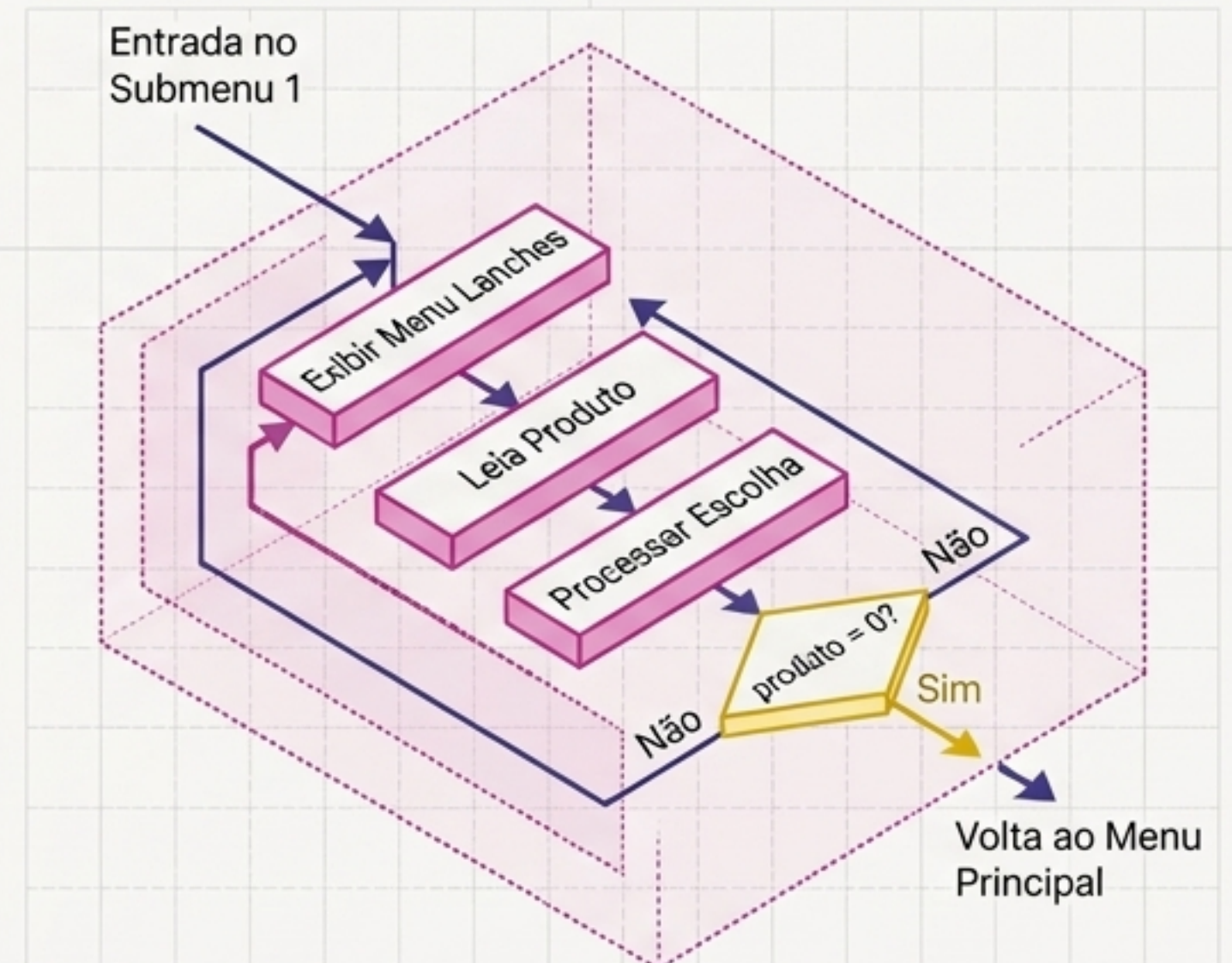
```
repita
// Exibição do Menu Principal
leia (opcao)
escolha opcao
  caso 1 // Vai para Lanches
  caso 2 // Vai para Acompanhamentos
  caso 3 // Vai para Bebidas
ate (opcao = 0)
```



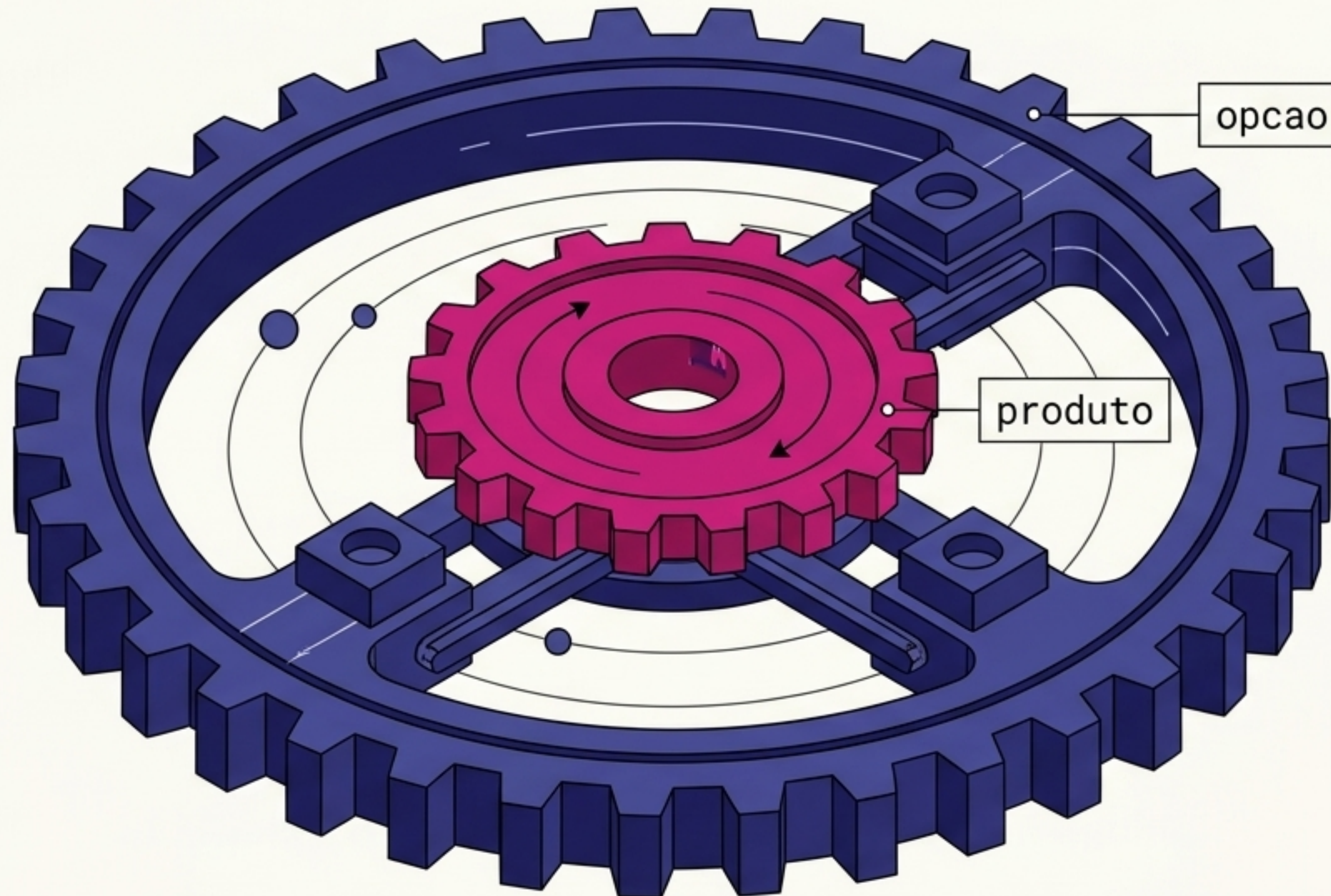
Os Laços Internos: Navegando nos Submenus

O usuário entra em um micro-universo (o menu de lanches). Ele pode comprar quantos X-saladas ou Coxinhas quiser. O sistema só o devolve ao menu principal quando ele digita 0.

```
caso 1
  repita
    escreval ('MENU DE LANCHES')
    escreval ('1 - X-salada - R$ 20,90')
    // ... outras opções ...
    escreval ('0 - voltar')
    leia(produto)
    // Lógica de preço e carrinho
  ate (produto = 0)
```



Laços Aninhados: Engrenagens Sincronizadas



A Regra de Ouro:

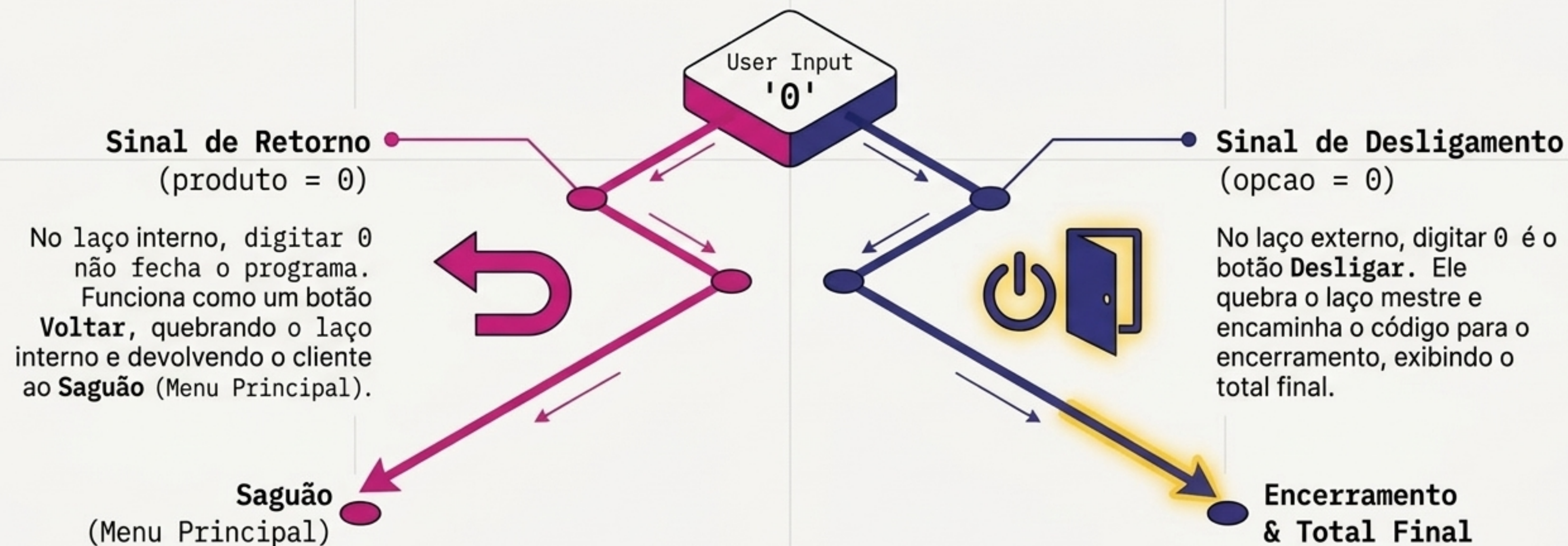
O laço interno deve terminar completamente o seu ciclo (até `produto = 0`) antes que o controle retorne ao laço externo.

Isolamento:

Variáveis de controle separadas (`opcao` para o externo, `produto` para o interno) evitam que as engrenagens travem umas às outras.

A Rota de Fuga: A Condição 0

A condição de saída no repita...ate é literalmente a chave para evitar laços infinitos que travam a máquina.

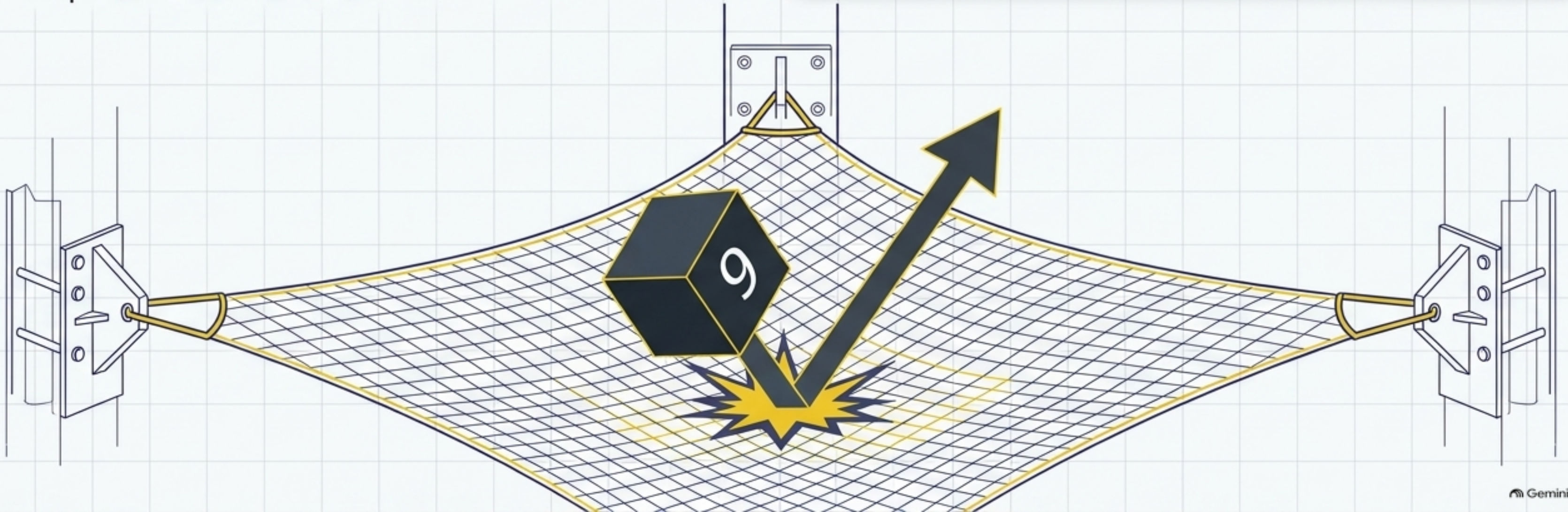


Tratamento de Exceções: A Rede de Segurança

O laço repita...ate é perfeito para lidar com erros de digitação. Se o usuário digitar um número fora do menu, o laço simplesmente engole o erro e repete a pergunta.

Sem o laço, um erro de digitação quebraria o algoritmo. Com o laço, o software se torna resiliente e à prova de falhas humanas.

```
se (produto <> 0) e (produto >= 1) e (produto <= 7) entao
    // Processa o pedido
senao
    se produto <> 0 entao
        escreval ('opcao invalida! digite 0 - 7')
```



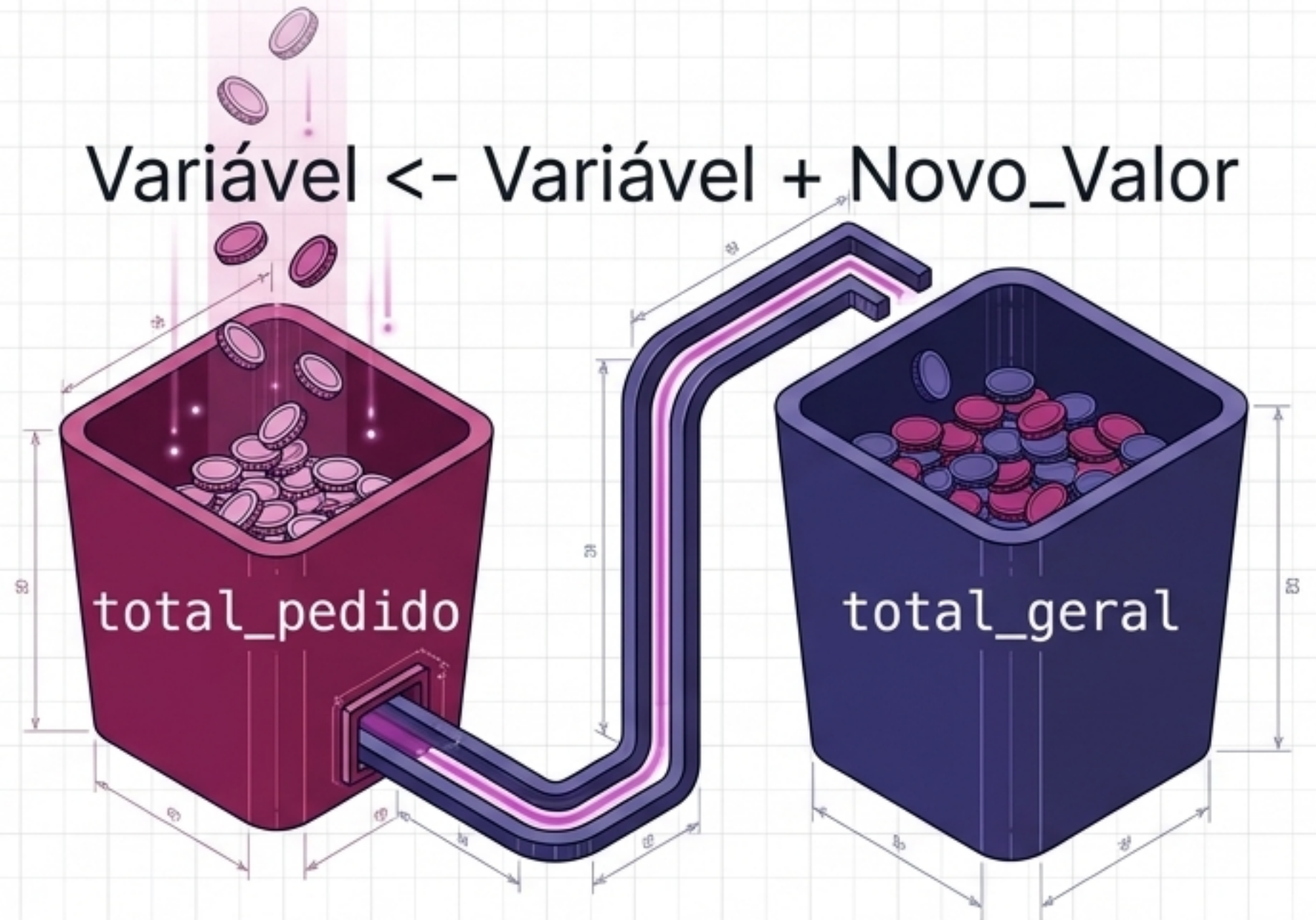
A Matemática do Laço: Acumuladores

Como o sistema lembra o que foi comprado em iterações anteriores?

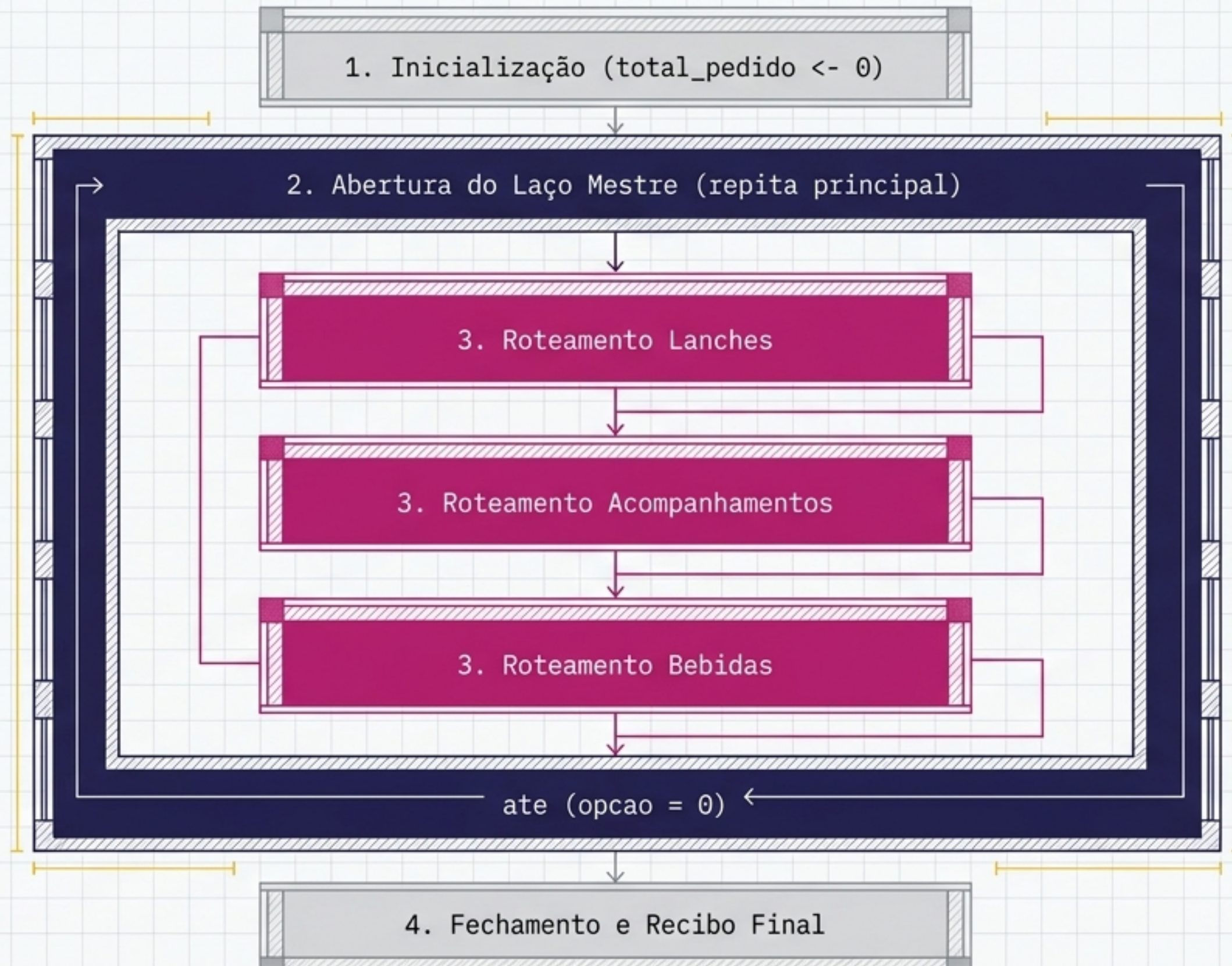
A cada giro do laço, o programa pega o valor que já estava na variável e soma com o custo do novo item.

Sem estar dentro de uma estrutura de repetição, essa conta substituiria o valor anterior em vez de acumulá-lo.

```
total_geral <- total_geral +  
preco * quantidade
```



O Fluxo Completo: Lendo a Matriz





O Domínio da Lógica Computacional

Dominar as estruturas de repetição (enquanto, repita...ate, para) não é apenas aprender a sintaxe de uma linguagem. É adquirir o poder de controlar o tempo e o fluxo dentro da máquina.

Com laços bem arquitetados e aninhados, você não escreve apenas scripts lineares; você constrói mundos digitais, sistemas persistentes e aplicativos do mundo real. O código da Lanchonete é apenas o começo da sua jornada como Arquiteto de Software.

